Project Hand-over Documentation

RDBL(Receipt,Disbursement,Banking,and Logs) System

1. Project Overview

The **RDBL System** is a web-based application designed to manage and monitor financial transactions, including receipts, disbursements, banking activities, and logging operations. It provides centralized tracking, reporting, and visualization of financial data to support operational and executive decision-making.

2. Project Scope

The system covers the following functional areas:

- Card Management
- Log Form Processing
- Log History Management
- Accounting Logs
- Logs Reporting
- Executive Dashboard (Cost Center & Card Expenses)
- Statement of Account Management and Creation

3. System Features & Modules

3.1 Card Management

- Create, update, and manage card details
- Assign cards to users
- Track card usage and associated expenses

3.2 Log Form

- Input form for recording financial transactions
- Supports Receipt, Disbursement, and Banking entries
- Includes validation and draft-saving functionality

3.3 Log History Management

- Tracks all approved and for approval logs
- Displays historical transaction records
- Supports filtering and searching

3.4 Accounting Logs

- Consolidated view of all finalize Statement Of Account(SOA)
- Update Reimbursement Details
- Used for accounting reconciliation

3.5 Logs Report

- Generates reports based on selected filters (date, type, etc.)
- Export functionality (e.g., Excel)
- Summary and detailed reporting views

3.6 Executive Dashboard

- Visual representation of:
    - Cost Center expenses
    - Card expenses
- Provides insights using charts and summarized data

3.7 Statement Management and Creation

- Generates financial statements
- Aggregates logs into structured reports
- Supports review and approval workflows
-

4. Technology Stack

Front-end

- HTML
- CSS
- JavaScript
- Bootstrap
- Tailwind CSS

Back-end

- Java

- Spring Framework (Spring MVC)

Database

- MySQL

ORM

- Hibernate

Architecture

- MVC (Model-View-Controller)

4. System Architecture Overview

- **Model Layer:** Handles database entities and business logic (Hibernate ORM).
- **View Layer:** JSP/HTML pages styled with Bootstrap and Tailwind CSS.
- **Controller Layer:** Spring MVC Controllers managing request flow and responses.

5. Key Features

- Dynamic form handling with validations
- Modular management system (scalable)
- Role-based access (if implemented)

6. Database Overview

- Relational database using MySQL
- Key tables include:
    - rbdl_log
    - rbdl_card_registration
    - user
    - rbdl_cost_center

- rbdl_soa

7. Setup Instructions

7.1 Prerequisites

- Java JDK (version 8)
- Apache Tomcat (or compatible server)
- MySQL Server
- IDE (Spring Tool Suite)

7.2 Steps to Run (Using SVN Repository)

1. **Access SVN Repository**
    - Open your IDE (e.g., Spring Tool Suite).
    - Navigate to SVN Repository Exploring perspective.
    - Enter the repository URL and log in using your assigned SVN username and password.
2. **Checkout Project**
    - Locate the **RBDL** project in the SVN repository.
    - Right-click the project and select **Checkout**.
    - Choose the appropriate workspace directory.
3. **Import Project into IDE**
    - If not automatically imported, go to:
        - *File → Import → Existing Projects into Workspace*
    - Select the checked-out project folder.
4. **Configure Database Connection**
    - Update database credentials in **WEB-INF > spring > properties > :**
        - db.local.properties
    - Ensure MySQL service is running.

5. **Initialize Database**
    - Run the provided SQL scripts to create required tables and initial data.
6. **Update Project Dependencies**
    - Right-click project → Maven → *Update Project* (if Maven-based).
    - Ensure all dependencies are downloaded successfully.
7. **Build and Clean Project**
    - Perform *Project → Clean* and rebuild the application.
8. **Deploy to Server**
    - Configure Apache Tomcat in your IDE.
    - Right-click project → *Run on Server* or manually add it to Tomcat.
9. **Access the Application**
    - Open browser and navigate to: http://localhost:8080/RBDL
10. **SVN Best Practices**

- Always **Update (Pull)** latest changes before starting work.
- Commit changes with proper comments.
- Resolve conflicts carefully before committing.

8. Configuration Details

- Database credentials located in:
    - db.local.properties
- Context path configurable in server settings
- Environment variables (if any) should be updated accordingly

9. Known Issues / Notes

- Ensure proper handling of null values in Hibernate mappings.
- Lazy loading may cause LazyInitializationException if not handled properly.
- Front-end responsiveness may require minor adjustments for smaller screens.
- Validation scripts are partly handled in JavaScript—review before modification.

10. Maintenance Guidelines

- Follow MVC pattern when adding new features.
- Ensure proper validation on both front-end and back-end.
- Use Hibernate best practices for database interactions.
- Maintain consistent naming conventions.
- Test all modules after any update.

11. Contact / Support

For any clarifications during the transition:

- Coordinate with previous developer or project owner.
- Review codebase comments and existing documentation.

12. Conclusion

This document provides a comprehensive overview of the **RDBL System**. The system is modular, scalable, and designed for operational efficiency. Proper understanding of each module and adherence to the architecture will ensure smooth maintenance and future development.

Project Documentation

Project Name: RDBL

Date: *May 06, 2026*

Created by: *John Rommel R. Rovero*